SmallCall: Post-Training a Small LLM for Tool-Calling

Why this project

Agentic AI systems in production depend on small, cheap models reliably deciding **when** to call a tool, **which** tool, and **when to call nothing at all**. Frontier models (GPT-4-class, 70B+) handle this well but are expensive to run at scale for every agent turn. The industry pattern — used by NVIDIA's Data Flywheel Blueprint, OpenPipe's ART-E, and function-calling leaderboards like BFCL — is to **post-train a small model on a narrow skill** so it can replace the expensive model for that specific job, and to **measure the gap that closes**, not just report a final accuracy number.

SmallCall reproduces that pattern end to end: baseline a small model against a larger reference, find its specific weakness, fix it with targeted fine-tuning, and prove the fix with before/after numbers on held-out data.

Keywords covered: LLM post-training, SFT (supervised fine-tuning), LoRA / PEFT, function-calling / tool-calling, held-out generalization testing, BFCL-style evaluation methodology, xLAM function-calling dataset, resumable/checkpointed training, Qwen2.5 model family, Hugging Face + Colab GPU training, quantitative before/after evaluation, failure taxonomy analysis.

Process

Stage 0 — Data and task design. Built the evaluation set from the xLAM function-calling dataset. Converted tool schemas to the format a model actually sees. Held out 15% of all tools entirely from training, so the test set measures true generalization to unseen tools, not memorization. Separately synthesized a "no suitable tool" test set by stripping the correct tool from examples where one existed — so the model is explicitly tested on knowing when **not** to call anything, a failure mode most tool-calling benchmarks ignore.

Final split sizes: 7,200 call examples + 800 no-call examples for training; 270+30 for validation; 400+150 for the held-out test set used in every result below.

Stage 1 — Baseline comparison. Ran the untrained Qwen2.5-1.5B model and a Qwen2.5-7B reference model on the identical 550-question test set, scored with both strict exact-match and a "fair" scorer that ignores cosmetic differences (default-valued or empty

arguments the model omits).

model	exact_fair_%	1_call_fair_%	2+*calls_fair*%	no_tool_correct_%	sec/example
Qwen2.5-1.5B (before)	84.5	87.2	79.3	76.0	0.372
Qwen2.5-7B (reference)	87.5	90.9	80.7	87.3	1.252

Diagnosis: the two models are close on calling accuracy (~3 points apart) but the small model is **11.3 points behind specifically on knowing when not to call a tool.** This became the defined, falsifiable target for fine-tuning — not a vague "improve tool-calling" goal.

Stage 2 — Targeted fine-tuning. LoRA (rank 16, applied to all attention and MLP projection matrices) supervised fine-tuning on the 8,000-example training set, 2 epochs, validated after each epoch to pick the best checkpoint by a blend of calling accuracy and no-tool accuracy. Trained with a resumable, checkpointed loop that survives Colab disconnects (environment issues included a torchao dependency conflict and a CUDA out-of-memory error from an oversized batch, both diagnosed and fixed mid-run).

Stage 3 — Final evaluation. Re-ran the identical 550-question held-out test with the fine-tuned model.

model	exact_fair_%	1_call_fair_%	2+*calls_fair*%	right_function_%	no_tool_correct_%	sec/example
Qwen2.5-1.5B (before)	84.5	87.2	79.3	98.2	76.0	0.372
Qwen2.5-1.5B (after SFT)	85.2	87.9	80.0	96.8	96.7	0.468
Qwen2.5-7B (reference)	87.5	90.9	80.7	98.8	87.3	1.252

Headline result: fine-tuning closed the 11.3-point refusal gap and then some — the 1.5B model's "no suitable tool" accuracy rose from 76.0% to 96.7%, surpassing the 7B reference's 87.3% — while overall calling accuracy held essentially flat (84.5% → 85.2%, no regression traded for the fix).

On the same 550 test questions: 13 previously-wrong calling questions fixed (10 newly wrong), and 31 previously-wrong no-tool questions fixed (0 newly wrong) — the fix was concentrated almost entirely where it was intended.

Stage 4 — Failure analysis. 64 failures remain out of 550 (fair scoring), broken down by type:

failure type	count
right function, wrong arguments	46
answered in text instead of calling	6
wrong number of calls	5
called a tool when none fit	5
broken call format	1
wrong function	1

The dominant remaining failure (46/64, ~72%) is argument extraction, not tool selection — a different, narrower problem than what this round of training targeted, and a natural next-step framing (e.g. argument-focused SFT or constrained decoding) rather than an open-ended weakness.

Resume / interview framing

"Post-trained a 1.5B model for tool-calling using LoRA SFT. Benchmarked it against a 7B reference model on a held-out-tool test set (BFCL-style methodology) and found the small model lagged specifically on refusal accuracy — correctly declining to call a tool when none fit (76% vs 87%). Closed that gap to 97%, beating the larger reference, with no loss in calling accuracy, using a resumable Colab training pipeline."

What this is not

This is a single round of SFT on one failure mode, not a full post-training pipeline — there was no RLHF/GRPO stage, and the remaining 46 argument-extraction errors are an open, named limitation rather than a solved problem. The secondary comparison against a larger model via the Groq API (gpt-oss-120b) was left incomplete (daily rate limit) and is explicitly out of scope for this result — the Qwen2.5-7B local reference is the one the headline numbers are measured against.